YuvaIntern Junior ReactJS Developer Internship

Week 1: Project Setup & Planning

ReactJS Project Plan Document

Name	Kaushal Kumar
Registration No.	24030485005
Institution	BIT Sindri, Jharkhand
Department	B.Tech Information Technology (2nd Year)
Internship	YuvaIntern – Junior ReactJS Developer
GitHub	github.com/kaushalvivek2005
LinkedIn	linkedin.com/in/kaushal-kumar-bb9506328
Date	June 2026

1. Executive Summary

This document presents the Week 1 deliverable for the YuvaIntern Junior ReactJS Developer Internship. The primary objective was to initialize a scalable ReactJS project from scratch, establish a well-organized directory structure, define team-wide coding standards, and integrate industry-standard code quality tooling before writing any feature code.

This planning-first approach mirrors real-world software engineering practice where proper setup directly determines the long-term maintainability of a codebase. A project that is well-architected from Day 1 significantly reduces technical debt in future weeks and makes collaboration easier.

2. Project Overview and Technology Choices

2.1 Development Approach

The project uses Create React App (CRA) as the build toolchain. CRA was chosen over a custom webpack setup for the following reasons:

- Zero-configuration: webpack, Babel, and Jest are pre-configured and battle-tested
- Faster iteration: no time wasted on build toolchain debugging in early weeks
- Ejectable: if custom webpack config is needed later, CRA can be ejected
- Industry adoption: widely used in professional React projects globally

2.2 Tool Selection Summary

Tool / Library	Version	Role	Justification
React	19.x	UI Library	Industry standard; component model; virtual DOM diffing
Create React App	5.0.1	Build Toolchain	Zero-config webpack + Babel; production-ready
ESLint	10.x	Static Code Analysis	Catches errors before runtime; enforces best practices
Prettier	3.x	Code Formatter	Eliminates style debates; auto-formats on save
Babel	7.x (via CRA)	JS Transpiler	Modern ES2021+ syntax in older browsers
React Context API	Built-in	State Management	Avoids Redux overhead for current project scope

React Router	v6 (Week 2)	Client Routing	Declarative routing; nested routes support

3. Directory Structure and Component Hierarchy

3.1 Planned Directory Tree

```
react-project/
├── public/
│   └── index.html              # Root HTML template
├── src/
│   ├── components/
│   │   ├── common/             # Atomic reusable components
│   │   │   ├── Button.jsx
│   │   │   ├── Input.jsx
│   │   │   └── Modal.jsx
│   │   ├── layout/             # App shell components
│   │   │   ├── Header.jsx
│   │   │   ├── Footer.jsx
│   │   │   └── Sidebar.jsx
│   │   └── pages/              # Route-level components
│   │       ├── HomePage.jsx
│   │       └── AboutPage.jsx
│   ├── hooks/                  # Custom React hooks
│   │   └── useLocalStorage.js
│   ├── context/                # Global state providers
│   │   └── AppContext.js
│   ├── services/               # API call layer
│   │   └── api.js
│   ├── utils/                  # Pure utility functions
│   │   └── helpers.js
│   ├── assets/
│   │   ├── images/             # Static image files
│   │   └── styles/             # Global CSS
│   └── constants/
│       └── index.js            # App-wide constants
├── .eslintrc.json
├── .prettierrc
├── .gitignore
└── package.json
```

3.2 Design Rationale

- components/common: stateless reusable atoms with no business logic; maximum reusability
- components/layout: rendered once per route, control overall page structure
- components/pages: composed from smaller components; one file per route
- hooks/: encapsulates stateful logic, keeps components thin and focused on rendering
- context/: global state (auth, theme) shared without prop drilling
- services/: all API communication isolated from the UI layer; easier to mock in tests

- utils/: pure functions with no React dependencies; independently unit-testable

4. Coding Standards and Naming Conventions

Type	Convention	Example
React Components	PascalCase	UserCard.jsx, HomePage.jsx
Custom Hooks	camelCase with use prefix	useLocalStorage.js, useAuth.js
Utility Functions	camelCase	helpers.js, formatDate()
Constants (values)	UPPER_SNAKE_CASE	API_ENDPOINTS, APP_NAME
CSS Classes	BEM methodology	nav__link--active, card__title
Context files	PascalCase + Context	AppContext.js, AuthContext.js
Test files	Same name + .test	Button.test.jsx, helpers.test.js

4.1 ESLint Configuration (.eslintrc.json)

```
{
  "env": { "browser": true, "es2021": true, "node": true },
  "extends": ["react-app", "plugin:react/recommended", "prettier"],
  "rules": {
    "react/react-in-jsx-scope": "off",
    "no-unused-vars": "warn",
    "no-console": "warn",
    "react-hooks/rules-of-hooks": "error",
    "react-hooks/exhaustive-deps": "warn"
  }
}
```

4.2 Prettier Configuration (.prettierrc)

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 80
}
```

5. Environment Setup Evidence — Code Snippets

5.1 Button Component (src/components/common/Button.jsx)

```
const Button = ({ label, onClick, variant = 'primary', disabled = false }) => {
  return (
    <button className={`btn btn--${variant}`} onClick={onClick} disabled={disabled}>
      {label}
    </button>
  );
};
export default Button;
```

5.2 Custom Hook (src/hooks/useLocalStorage.js)

```
const useLocalStorage = (key, initialValue) => {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) { return initialValue; }
  });
  const setValue = value => {
    setStoredValue(value);
    window.localStorage.setItem(key, JSON.stringify(value));
  };
  return [storedValue, setValue];
};
```

5.3 NPM Scripts in package.json

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "lint": "eslint src/ --ext .js,.jsx",
  "lint:fix": "eslint src/ --ext .js,.jsx --fix",
  "format": "prettier --write src/**/*.{js,jsx,css}"
}
```

6. Development Roadmap

Week	Focus	Key Deliverables	Status
Week 1	Setup & Planning	CRA init, ESLint, Prettier, directory structure, project plan document	Completed
Week 2	Reusable UI Component Library	5+ reusable components (Button, Form, Modal, Card, Navbar), component documentation	Upcoming

Week 3	State Management & Hooks	Task manager mini-app, useState, useEffect, custom hook, Context API, documentation	Upcoming
Week 4	API Integration & Data Handling	Public API integration, async fetch, error handling, loading states, report	Upcoming
Week 5	Testing & Debugging	Jest + React Testing Library, unit tests, integration tests, testing report	Upcoming
Week 6	Performance & Accessibility	Lighthouse audit, lazy loading, code splitting, ARIA attributes, WCAG compliance report	Upcoming

7. GitHub Repository and Version Control

The project is hosted on GitHub and follows a structured workflow from Day 1. Version control is treated as a core part of development, not an afterthought.

7.1 Repository Details

- Repository URL: https://github.com/kaushalvivek2005/react-project
- Branch: main (protected, production-ready code only)
- Visibility: Public

7.2 Commit Convention (Conventional Commits)

```
feat: add Button component with variant and disabled support
fix: resolve ESLint warning on useEffect exhaustive deps
chore: configure Prettier, ESLint, and directory structure
docs: update README with setup and usage instructions
```

8. Conclusion

Week 1 has successfully delivered a production-grade project foundation for the ReactJS internship. Key outcomes include: a fully initialized React 19 project using Create React App, a logical directory structure following separation of concerns, ESLint and Prettier configured for consistent code quality, and sample components demonstrating the intended architectural patterns.

The time invested in planning, research, and setup this week will compound into significant time savings in future weeks. A clear architecture and coding standard eliminates the most common sources of

confusion and refactoring in early-stage projects. All deliverables are version-controlled on GitHub and this document serves as the authoritative reference for all technical decisions made during Week 1.

— End of Document —